

Spring Boot Auto-configuration Internals

A Deep Dive into How Spring Boot Works Under the Hood

Chapter 05 -- Spring Deep Dive Series

A. Spring Boot vs Spring	Starters, auto-config, embedded server
B. @SpringBootApplication	@Configuration + @ComponentScan + @EnableAutoConfiguration
C. Auto-config Loading	AutoConfiguration.imports, AutoConfigurationImportSelector
D. @Conditional Annotations	OnClass, OnMissingBean, OnProperty
E. DataSourceAutoConfiguration	Real walk-through of DataSource bean creation
F. Properties Binding	@ConfigurationProperties, @Value, property hierarchy
G. SpringApplication.run()	prepareEnvironment -> createContext -> refreshContext
H. Embedded Tomcat	TomcatServletWebServerFactory, DispatcherServlet
I. Custom Auto-configuration	Writing your own with @AutoConfiguration
J. Spring Boot Actuator	/health, /metrics, /beans internals

1. Plain English Explanation

Spring Framework is a powerful but verbose toolkit. To build a web app you had to manually configure a DispatcherServlet, a DataSource, a transaction manager, a view resolver, deploy a WAR to an external Tomcat, and write dozens of XML or Java config files. Spring Boot eliminates this ceremony. It gives you four things on top of plain Spring:

- **Starters** -- curated Maven/Gradle dependency bundles (spring-boot-starter-web pulls Tomcat + Spring MVC + Jackson).
- **Auto-configuration** -- detects what is on the classpath and wires sensible beans automatically.
- **Embedded server** -- Tomcat, Jetty, or Undertow runs inside your JAR; no external deployment needed.
- **Opinionated defaults** -- sane property values and conventions so 80% of apps need zero config.

2. Real-World Analogy

Think of Spring Framework as a fully equipped kitchen with every utensil available -- but bare cupboards. You must stock it yourself. Spring Boot is a meal-kit delivery service: the right ingredients arrive pre-measured, the oven preheats automatically, and a recipe card covers 95% of common meals. You can still swap ingredients, but you rarely need to.

3. Minimal Working Example

Plain Spring (pre-Boot) required explicit servlet config; Spring Boot collapses this to:

```
// ONE file, ONE annotation -- that is all Spring Boot needs
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}

// pom.xml dependency -- the starter pulls 30+ transitive jars
// <dependency>
//   <groupId>org.springframework.boot</groupId>
//   <artifactId>spring-boot-starter-web</artifactId>
// </dependency>
```

4. Architecture Diagram

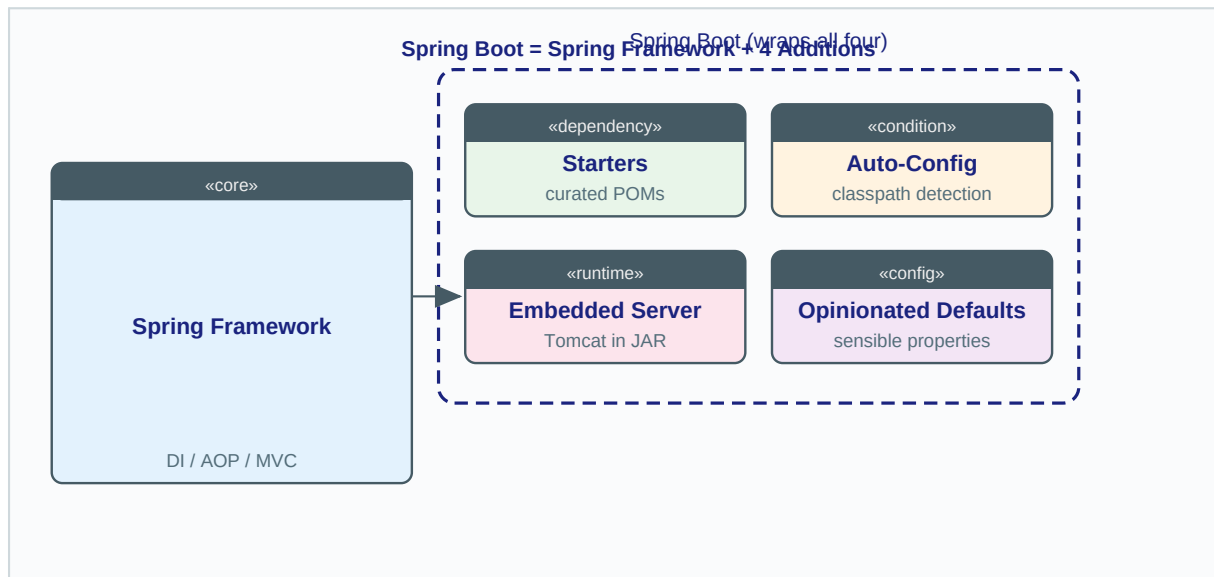


Figure A-1: Spring Boot wraps Spring Framework with four key additions

5. Behind the Scenes

When you add `spring-boot-starter-web` to your POM, Maven resolves the dependency tree transitively: `spring-webmvc`, `spring-web`, `spring-context`, `jackson-databind`, `tomcat-embed-core`, and more -- about 30 jars. The starter itself contains no code; it is a pure POM. At startup, `SpringApplication` inspects the classpath and delegates to `AutoConfigurationImportSelector` to decide which `@Configuration` classes to activate.

6. Realistic Practical Example

```
// spring-boot-starter-data-jpa pulls in:
//  Hibernate, Spring Data JPA, Spring ORM, HikariCP, spring-tx
//  No XML. No explicit bean definitions. Just add the starter:
@Entity
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
}

@Repository
public interface ProductRepo
    extends JpaRepository<Product, Long> {}
// Spring Data auto-generates the SQL implementation at runtime
```

7. Common Mistakes and Misconceptions

Mistake: Adding both `spring-boot-starter-web` and `spring-boot-starter-webflux` -- this causes conflicts because Boot cannot auto-configure both reactive and servlet stacks simultaneously.

Misconception: Starters lock you in. In reality, you can exclude any auto-configured bean with `@SpringBootApplication(exclude=...)` or by defining your own bean.

Good practice: Use the exact starter for your use case (`starter-web`, `starter-data-jpa`, `starter-security`) rather than adding transitive jars manually.

1. Plain English Explanation

@SpringBootApplication is a convenience meta-annotation. It combines three annotations that almost every Spring Boot application needs. Understanding each piece explains exactly what Boot does at startup:

- **@Configuration** -- marks the class as a source of @Bean definitions processed by Spring's container.
- **@ComponentScan** -- tells Spring to scan the package (and sub-packages) for @Component, @Service, @Repository, @Controller classes.
- **@EnableAutoConfiguration** -- triggers Boot's auto-configuration machinery to load condition-based configuration classes.

2. Real-World Analogy

Think of it as a Swiss Army knife. You could carry three separate tools (@Configuration, @ComponentScan, @EnableAutoConfiguration) but Spring Boot hands you a single combined tool. You can still pull out individual blades -- for instance, disabling component scan by replacing the combined annotation with the individual three.

3. Minimal Working Example

```
// What @SpringBootApplication actually expands to:
@Configuration
@ComponentScan(basePackages = "com.example")
@EnableAutoConfiguration
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
// The combined annotation is just shorthand -- both are identical.
// You can use the three individual annotations when you need
// fine-grained control (e.g., custom basePackageClasses).
```

4. Architecture Diagram

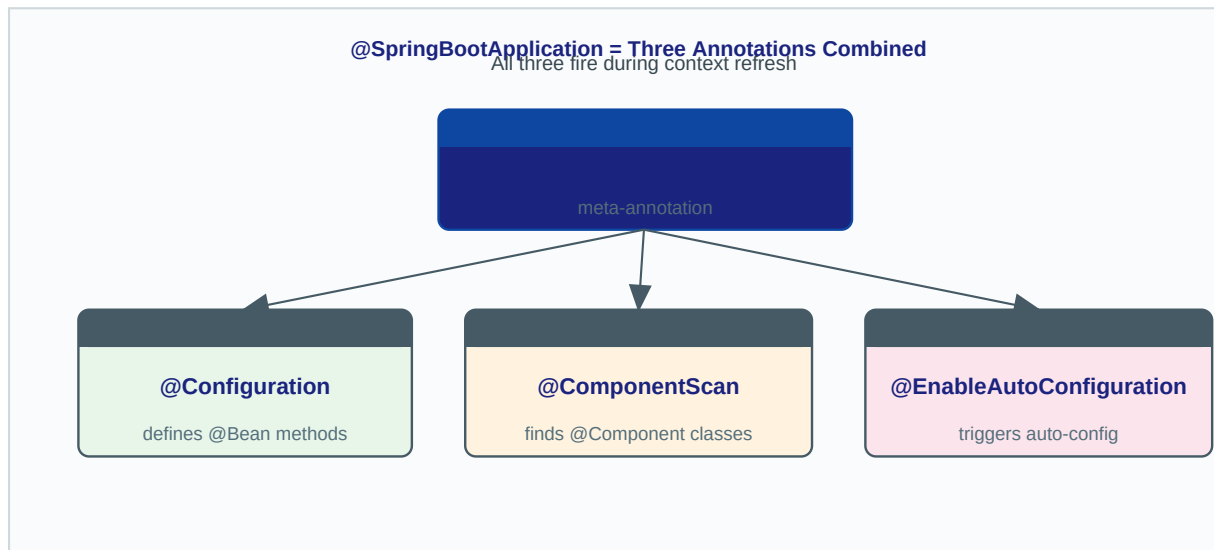


Figure B-1: `@SpringBootApplication` decomposes into three distinct annotations

5. Behind the Scenes

Spring processes `@SpringBootApplication` via `AnnotationConfigUtils`. The `@Configuration` marker registers the class with `ConfigurationClassPostProcessor` -- a `BeanFactoryPostProcessor` that parses `@Bean` methods. `@ComponentScan` triggers `ClassPathBeanDefinitionScanner` which walks the classpath looking for classes annotated with stereotype annotations. `@EnableAutoConfiguration` is an `@Import` pointing to `AutoConfigurationImportSelector` -- the entry point to Boot's auto-config mechanism (covered in Concept C).

6. Realistic Practical Example

```
// Splitting @SpringBootApplication when you need control:
@Configuration
@EnableAutoConfiguration
@ComponentScan(basePackageClasses = {
    AppConfig.class,
    SecurityConfig.class
})
public class App {
    // Custom basePackageClasses limits scan to specific roots
    // Useful in modular monolith or multi-module Maven projects
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: Placing the main class deep in a sub-package -- `@ComponentScan` only scans the current package and below, so beans in sibling packages are missed.

Mistake: Adding `@ComponentScan` on multiple classes in the same app -- this causes duplicate bean registration exceptions.

Good practice: Keep the `@SpringBootApplication` class at the root package (e.g., `com.example`) so all sub-packages are automatically scanned.

C

How Auto-configuration is Loaded

AutoConfiguration.imports | AutoConfigurationImportSelector | SpringFactoriesLoader

1. Plain English Explanation

When Spring Boot starts, it needs to know which auto-configuration classes to activate. It uses a registry file -- META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports -- that lists every available auto-config class, one per line. AutoConfigurationImportSelector reads this file and returns the class names to Spring's context, which then evaluates each class's @Conditional annotations to decide whether to actually apply it.

2. Real-World Analogy

Imagine a hotel concierge (AutoConfigurationImportSelector) holding a master list of every available hotel service (the .imports file). When you check in, the concierge reads the list and asks: Does this guest need the gym? Is the spa relevant? Each service decides for itself whether to activate based on the guest's profile (@Conditional).

3. Minimal Working Example

```
// META-INF/spring/  
// org.springframework.boot.autoconfigure.AutoConfiguration.imports  
// (inside spring-boot-autoconfigure.jar -- excerpt):  
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration  
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration  
org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration  
  
// Spring Boot 2.7+ uses this file; older versions used  
// META-INF/spring.factories with key:  
// org.springframework.boot.autoconfigure.EnableAutoConfiguration= // com.example.MyAutoConfig  
uration
```

4. Architecture Diagram

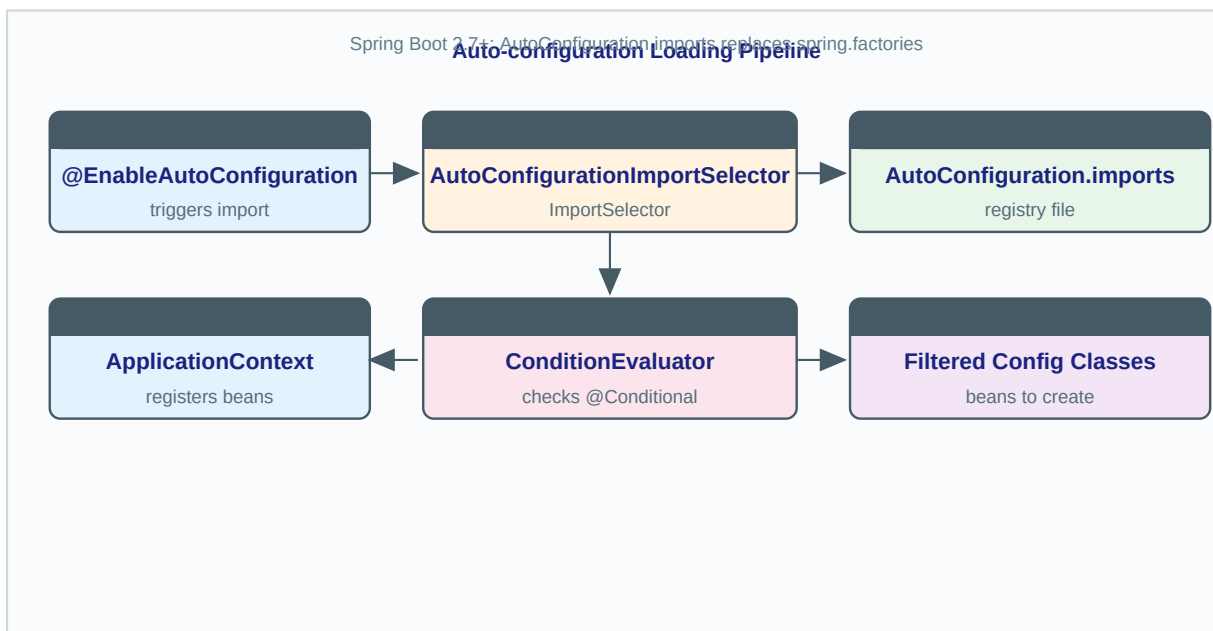


Figure C-1: The auto-configuration loading pipeline from annotation to bean

5. Behind the Scenes

`@EnableAutoConfiguration` is meta-annotated with `@Import(AutoConfigurationImportSelector.class)`. `AutoConfigurationImportSelector` implements `DeferredImportSelector`, which means it runs after all regular `@Configuration` classes have been processed. Its `selectImports()` method calls `getAutoConfigurationEntry()` which invokes `SpringFactoriesLoader.loadFactoryNames()` (Spring Boot 2.x) or `ImportCandidates.load()` (Spring Boot 3.x) to read the `.imports` file from every JAR on the classpath. The full list is then deduplicated and filtered through `OnClassCondition`, `OnBeanCondition`, and `OnPropertyCondition` before being returned to Spring.

6. Realistic Practical Example

```
// To register your own auto-configuration (Spring Boot 3.x):
// File: src/main/resources/META-INF/spring/
//   org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.mylib.MyLibAutoConfiguration

// The auto-configuration class itself:
@AutoConfiguration
@ConditionalOnClass(MyService.class)
public class MyLibAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    public MyService myService() {
        return new MyService();
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: Using `spring.factories` in a Spring Boot 3.x project -- the key `org.springframework.boot.autoconfigure.EnableAutoConfiguration` is no longer read; you must use the `.imports` file.

Misconception: All listed auto-config classes always run -- in reality, most are skipped by `@Conditional` checks; only relevant ones activate.

Good practice: Use `spring-boot-autoconfigure-processor` annotation processor to generate auto-config metadata, which speeds up condition evaluation at startup.

1. Plain English Explanation

@Conditional annotations are Spring Boot's gatekeeping mechanism. They let a @Configuration class or @Bean method declare: 'only create me if a certain condition is true.' Three conditions dominate Boot's own auto-configurations:

- **@ConditionalOnClass** -- activate only if a specific class is present on the classpath.
- **@ConditionalOnMissingBean** -- activate only if no bean of that type has been defined yet (user-defined beans take priority).
- **@ConditionalOnProperty** -- activate only if a specific property key exists (and optionally has a given value).

2. Real-World Analogy

A smart home hub checks sensor readings before acting: 'Turn on heating only if temperature is below 18C (OnProperty). Use the Nest thermostat if one is detected (OnClass). Skip setup if the user already configured heating manually (OnMissingBean).' Auto-configuration follows exactly the same if-then logic.

3. Minimal Working Example

```
@Configuration
@ConditionalOnClass(DataSource.class)    // only if JDBC driver is present
public class MyDataConfig {

    @Bean
    @ConditionalOnMissingBean             // skip if user defined their own
    public DataSource dataSource() {
        return DataSourceBuilder.create()
            .url("jdbc:h2:mem:test")
            .build();
    }

    @Bean
    @ConditionalOnProperty(prefix = "app.cache",
                           name = "enabled",
                           havingValue = "true")
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager();
    }
}
```

4. Architecture Diagram

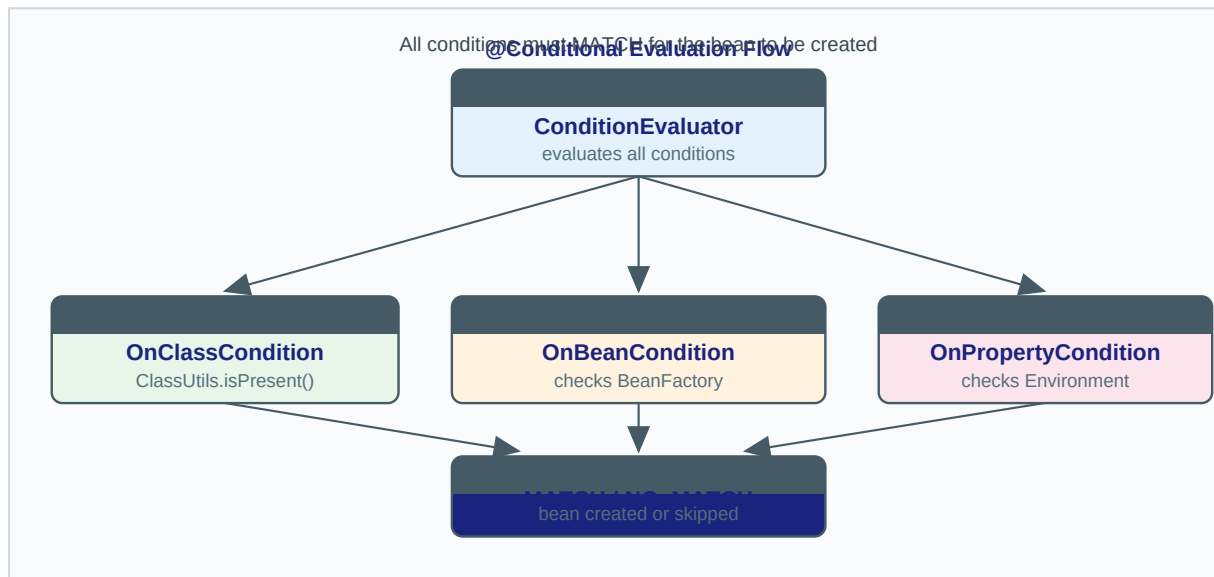


Figure D-1: ConditionEvaluator delegates to specific condition implementations

5. Behind the Scenes

Spring's `ConditionEvaluator.shouldSkip()` iterates the `@Conditional` annotations on a class or method. For `@ConditionalOnClass`, `OnClassCondition.getOutcome()` calls `ClassUtils.isPresent(className, classLoader)` -- a simple try/catch around `Class.forName()`. For `@ConditionalOnMissingBean`, `OnBeanCondition` queries the `BeanDefinitionRegistry` to see if any bean of the target type already exists. For `@ConditionalOnProperty`, `OnPropertyCondition` reads the Spring Environment (which aggregates all property sources). If any condition returns `NO_MATCH`, the bean definition is skipped entirely.

6. Realistic Practical Example

```
// Real Spring Boot: WebMvcAutoConfiguration snippet (simplified)
@Configuration(after = DispatcherServletAutoConfiguration.class)
@ConditionalOnWebApplication(type = Type.SERVLET)
@ConditionalOnClass({ Servlet.class, DispatcherServlet.class })
@ConditionalOnMissingBean(WebMvcConfigurationSupport.class)
public class WebMvcAutoConfiguration {
    // Only activates when:
    // 1. This is a servlet web app
    // 2. DispatcherServlet class is on classpath
    // 3. User has NOT defined their own WebMvcConfigurationSupport
    @Bean
    @ConditionalOnMissingBean(HiddenHttpMethodFilter.class)
    public OrderedHiddenHttpMethodFilter hiddenHttpMethodFilter() {
        return new OrderedHiddenHttpMethodFilter();
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: `@ConditionalOnMissingBean` checks only registered beans at condition evaluation time, not beans that will be registered later -- ordering matters.

Mistake: Using `@ConditionalOnClass` with a class that is always present (like `String.class`) -- the condition is always true and provides no value.

Good practice: Combine `@ConditionalOnClass` (guards against `ClassNotFoundException`) with `@ConditionalOnMissingBean` (lets users override) -- this is the standard Boot pattern.

1. Plain English Explanation

`DataSourceAutoConfiguration` is the canonical example of Boot's auto-config in action. It examines your classpath and properties to decide whether and how to create a `javax.sql.DataSource` bean. If you add any JDBC driver JAR plus `spring-boot-starter-data-jpa`, Boot will automatically wire a connection pool (HikariCP by default) using the URL you provide in `application.properties` -- without a single line of explicit bean configuration.

2. Real-World Analogy

It is like a water dispenser that detects the bottle type: if a 5-gallon jug is present, it wires up for floor-standing delivery; if no jug is present, it stays dormant. If you bring your own dispenser, Boot's default one stays on the shelf.

3. Minimal Working Example

```
// application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=admin
spring.datasource.password=secret
// That is all! DataSourceAutoConfiguration reads these and
// creates a HikariCP DataSource bean automatically.

// To override Boot's DataSource with your own:
@Bean
public DataSource dataSource() {
    // Boot detects this bean and skips its own configuration
    return new EmbeddedDatabaseBuilder()
        .setType(EmbeddedDatabaseType.H2).build();
}
```

4. Architecture Diagram

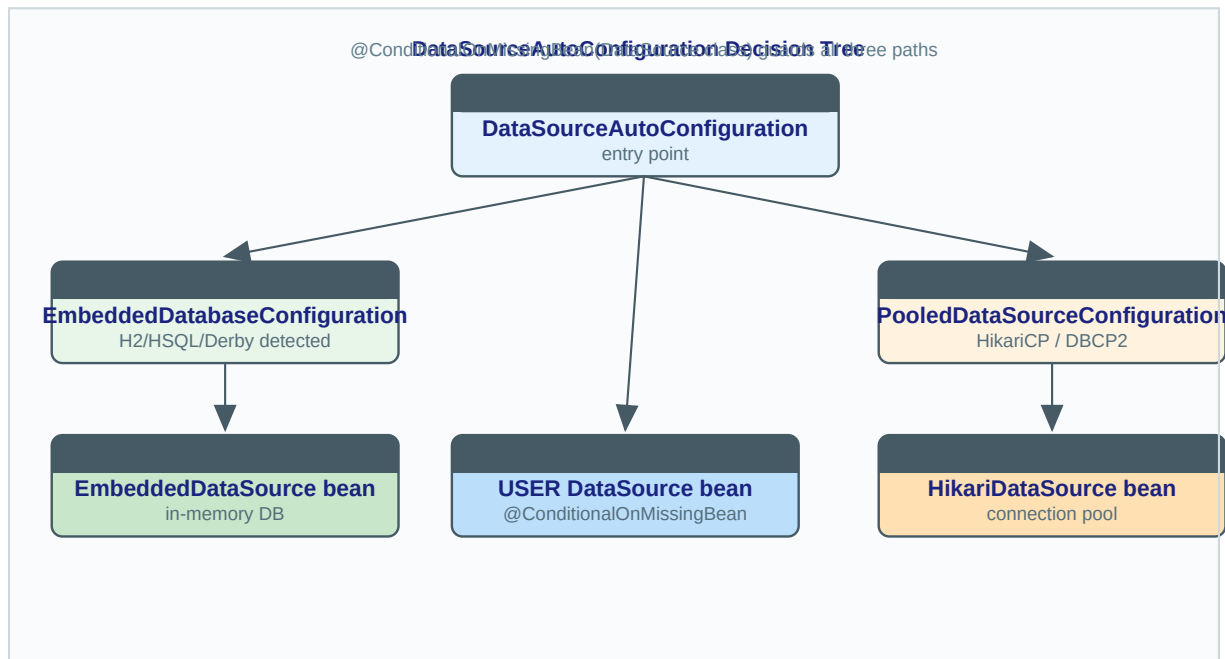


Figure E-1: DataSourceAutoConfiguration selects the right DataSource implementation

5. Behind the Scenes

DataSourceAutoConfiguration is annotated @ConditionalOnClass({DataSource.class, EmbeddedDatabaseType.class}) and @ConditionalOnMissingBean(DataSource.class). It imports DataSourceConfiguration which has inner classes for Hikari, Tomcat pool, DBCP2, and embedded databases, each guarded by @ConditionalOnClass for their specific pool class AND @ConditionalOnProperty(spring.datasource.type). HikariCP wins by default because its @ConditionalOnClass(HikariDataSource.class) matches when spring-boot-starter-jdbc is on the classpath. DataSourceProperties.class is bound via @ConfigurationProperties(prefix='spring.datasource') so url, username, and password flow automatically from application.properties into the pool builder.

6. Realistic Practical Example

```

// Custom DataSource with connection pool tuning --
// Boot's auto-config skips because @Bean is present:
@Configuration
public class DbConfig {
    @Bean
    public DataSource dataSource(
        @Value("${db.url}") String url,
        @Value("${db.user}") String user,
        @Value("${db.pass}") String pass) {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl(url);
        cfg.setUsername(user);
        cfg.setPassword(pass);
        cfg.setMaximumPoolSize(20);
        cfg.setConnectionTimeout(3000);
        return new HikariDataSource(cfg);
    }
}
  
```

7. Common Mistakes and Misconceptions

Mistake: Defining a DataSource bean but also setting spring.datasource.url -- the URL is now unused (your bean ignores it) and may cause confusion.

Mistake: Forgetting to add a JDBC driver dependency -- without it, @ConditionalOnClass(DataSource.class) may still pass (DataSource is in javax.sql, part of the JDK) but the connection will fail at runtime.

Good practice: Use spring.datasource.hikari.* properties to tune Hikari without needing a custom @Bean -- Boot's properties binding handles it.

1. Plain English Explanation

Spring Boot reads configuration from many places and assembles them into an ordered Environment. Your code accesses values via `@Value` (single property injection) or `@ConfigurationProperties` (bulk binding to a Java POJO). The property source hierarchy defines which source wins when the same key appears multiple times -- environment variables beat `application.properties`, which beats default values.

2. Real-World Analogy

Think of layers of sticky notes on a whiteboard. The top note (command-line arg) covers anything below. Remove it and the next one (env variable) shows. Each layer can add or override the layers beneath. Your `application.properties` is a middle layer -- useful as a default but easily overridden for production.

3. Minimal Working Example

```
// application.properties
app.service.timeout=30
app.service.base-url=https://api.example.com

// @ConfigurationProperties POJO:
@ConfigurationProperties(prefix = "app.service")
@Component
public class ServiceProps {
    private int timeout;
    private String baseUrl;    // kebab-case auto-mapped to camelCase
    // getters + setters required
}

// @Value injection (single property):
@Service
public class MyService {
    @Value("${app.service.timeout:15}") // 15 = default
    private int timeout;
}
```

4. Architecture Diagram

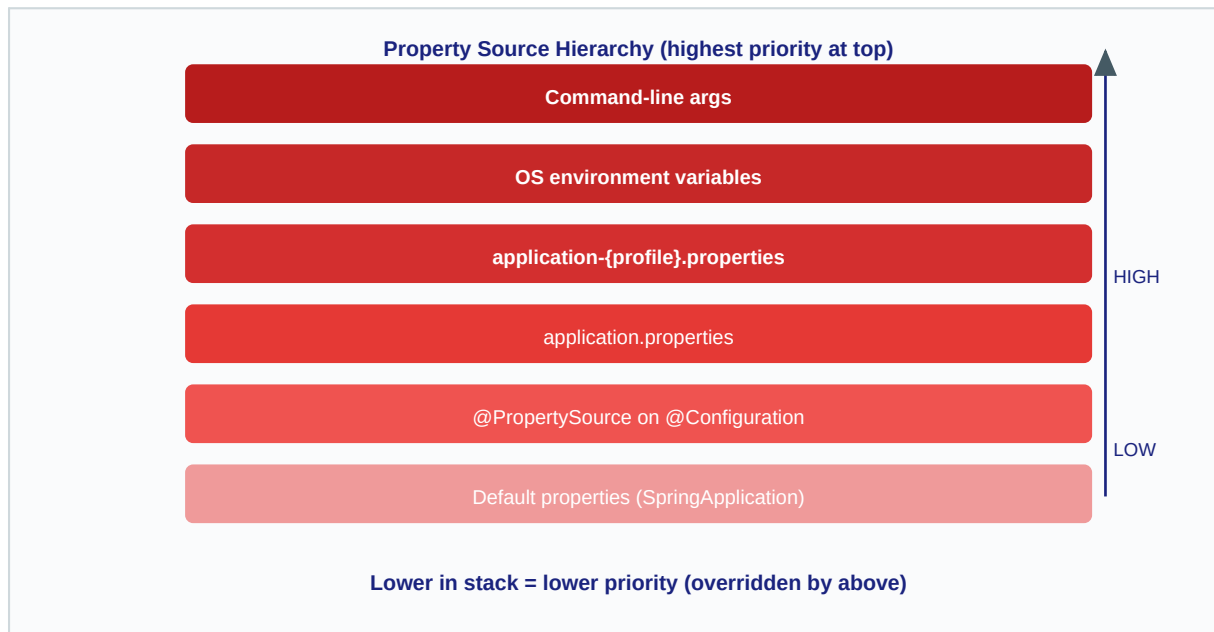


Figure F-1: Spring Boot property source hierarchy -- higher sources override lower ones

5. Behind the Scenes

@ConfigurationProperties works via ConfigurationPropertiesBindingPostProcessor, a BeanPostProcessor that intercepts beans annotated with @ConfigurationProperties. Its postProcessBeforeInitialization() method calls Binder.bind() which walks the Spring Environment's MutablePropertySources list in priority order. Binder uses reflection to map property names (using RelaxedPropertyResolver for kebab-case, camelCase, UPPER_SNAKE_CASE equivalence) to the POJO's setter methods. @Value injection is handled earlier by AutowiredAnnotationBeanPostProcessor via PropertyPlaceholderHelper which resolves \${...} expressions from the same Environment.

6. Realistic Practical Example

```
@ConfigurationProperties(prefix = "payment")
@Validated // enables JSR-303 validation on the POJO
public class PaymentConfig {
    @NotEmpty
    private String apiKey;
    @Min(1) @Max(300)
    private int timeoutSeconds = 30;
    private Map<String, String> endpoints = new HashMap<>();
    // Spring Boot binds payment.endpoints.visa=https://...
    // as a map entry automatically
}
// application.yml:
// payment:
//   api-key: sk-live-abc123
//   timeout-seconds: 45
//   endpoints:
//     visa: https://api.visa.com
```

7. Common Mistakes and Misconceptions

Mistake: Using @Value with a property that does not exist and no default -- Spring throws BeanCreationException at startup with a cryptic 'Could not resolve placeholder' message.

Misconception: `application.yml` and `application.properties` are identical -- YAML supports lists and nested structures more naturally, but both are loaded; if both exist, `.properties` wins in Boot 2.4+.

Good practice: Prefer `@ConfigurationProperties` over `@Value` for groups of related properties -- it gives you validation, IDE completion, and type safety.



SpringApplication.run() Startup Sequence

prepareEnvironment -> createApplicationContext -> refreshContext -> callRunners

1. Plain English Explanation

SpringApplication.run() is the single entry point that boots your entire application. It performs six major phases in strict order: bootstrap the listeners, prepare the environment, print the banner, create the ApplicationContext, refresh the context (which registers all beans and triggers auto-configuration), and finally call any ApplicationRunner or CommandLineRunner beans.

2. Real-World Analogy

Think of an airline pre-flight checklist: first load passengers (environment), then verify the manifest (context creation), then taxi and run engine checks (refresh -- wiring all beans), and finally announce the destination (runners). Each step must complete before the next begins.

3. Minimal Working Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        // This one call executes ALL six phases below
        SpringApplication.run(App.class, args);
    }

    // Called AFTER context is fully refreshed:
    @Bean
    public ApplicationRunner startupRunner() {
        return args -> System.out.println("App ready! " + args);
    }
}

// Order: prepareEnv -> createCtx -> prepareCtx
//         -> refreshCtx -> afterRefresh -> callRunners
```

4. Architecture Diagram

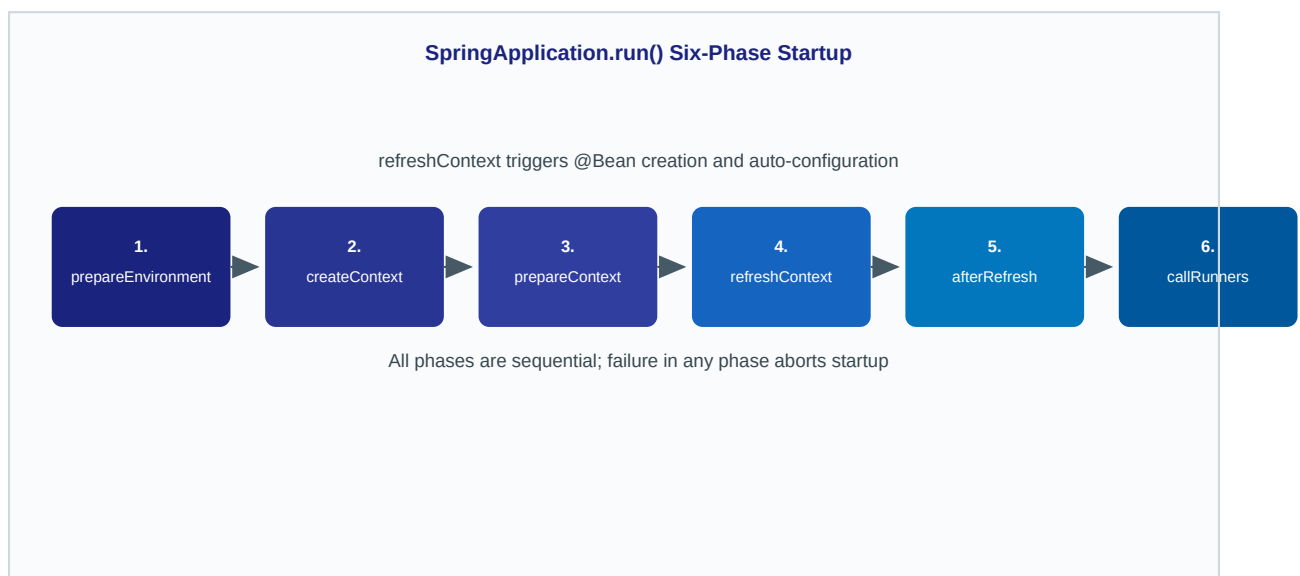


Figure G-1: The six sequential phases of `SpringApplication.run()`

5. Behind the Scenes

`prepareEnvironment()` loads all `PropertySource` implementations (system props, env vars, `application.properties`) into a `ConfigurableEnvironment` and fires an `ApplicationEnvironmentPreparedEvent`. `createApplicationContext()` uses `ApplicationContextFactory` to instantiate the correct context type -- `AnnotationConfigServletWebServerApplicationContext` for servlet apps. `prepareContext()` applies `ApplicationContextInitializers` and registers the primary configuration source. `refreshContext()` calls `AbstractApplicationContext.refresh()` which invokes `ConfigurationClassPostProcessor` (parses all `@Configuration` classes), registers `BeanDefinitions`, instantiates singletons, and starts the embedded server. `callRunners()` retrieves all `ApplicationRunner` and `CommandLineRunner` beans and invokes them.

6. Realistic Practical Example

```
@Component
public class DatabaseSeeder implements ApplicationRunner {
    private final UserRepository userRepo;
    public DatabaseSeeder(UserRepository userRepo) {
        this.userRepo = userRepo;
    }
    @Override
    public void run(ApplicationArguments args) {
        if (userRepo.count() == 0) {
            userRepo.save(new User("admin", "admin@example.com"));
            System.out.println("Seeded initial admin user");
        }
    }
    // Runs after ALL beans are wired -- safe to use any injected dependency
}
```

7. Common Mistakes and Misconceptions

Mistake: Running expensive logic in a `@PostConstruct` method instead of `ApplicationRunner` --

`@PostConstruct` fires during bean initialization inside `refreshContext`, before all beans are ready.

Misconception: `SpringApplication.run()` is synchronous and blocks -- it does block the main thread until all runners finish, but the embedded server starts in a background thread during refresh.

Good practice: Use `@Order` on multiple `ApplicationRunner` beans to control execution sequence.

1. Plain English Explanation

Traditional Spring MVC required deploying a WAR file to an external Tomcat server. Spring Boot flips this: Tomcat is embedded inside your JAR and starts programmatically as part of the `ApplicationContext` refresh. When the context starts, it creates a `TomcatWebServer`, registers the `DispatcherServlet`, and opens a port -- all automatically.

2. Real-World Analogy

Old-school deployment is like shipping cargo to a warehouse and letting the warehouse decide how to handle it. Embedded Tomcat is like a food truck: the kitchen (Tomcat) is built into the vehicle (your JAR). You drive to any location and start serving immediately -- no warehouse setup required.

3. Minimal Working Example

```
// Customise embedded Tomcat without XML:
@Bean
public TomcatServletWebServerFactory tomcatFactory() {
    TomcatServletWebServerFactory factory =
        new TomcatServletWebServerFactory();
    factory.setPort(8443);
    factory.addConnectorCustomizers(connector -> {
        connector.setScheme("https");
        connector.setSecure(true);
    });
    return factory;
}
// Or via properties (simpler):
// server.port=8443
// server.ssl.key-store=classpath:keystore.p12
```

4. Architecture Diagram

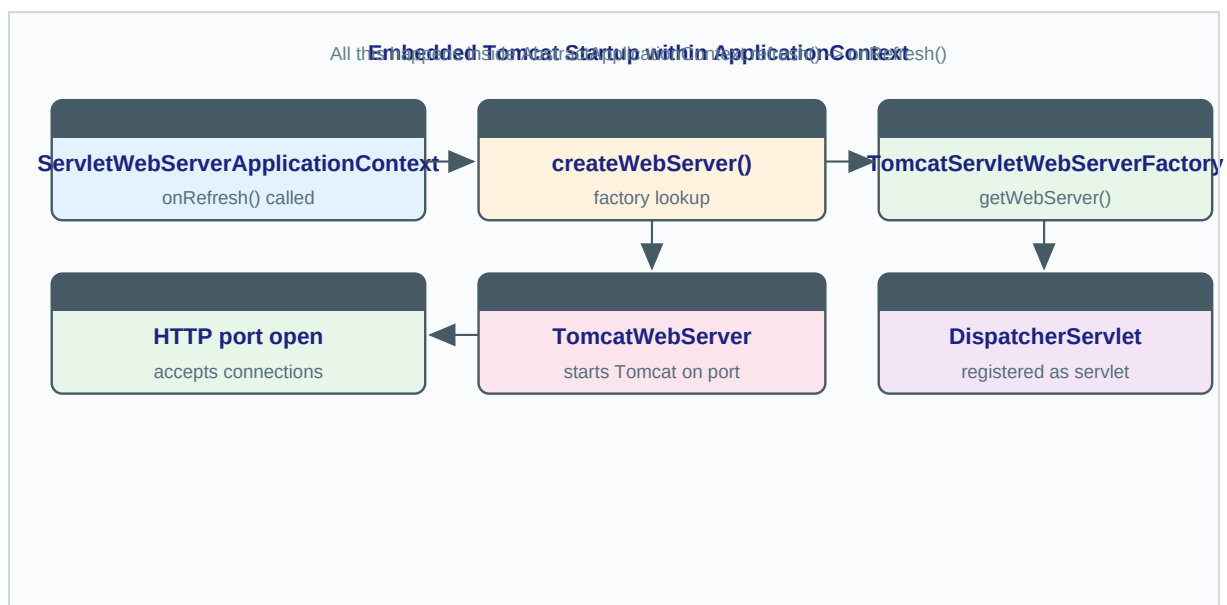


Figure H-1: Embedded Tomcat creation chain during context refresh

5. Behind the Scenes

`ServletWebServerApplicationContext` overrides `onRefresh()` -- a hook called at the end of `AbstractApplicationContext.refresh()`. It calls `createWebServer()` which looks up a `ServletWebServerFactory` bean (`TomcatServletWebServerFactory` by default, supplied by `EmbeddedWebServerFactoryCustomizerAutoConfiguration`). `getWebServer()` creates a `org.apache.catalina.startup.Tomcat` instance, configures it programmatically, and adds a `TomcatEmbeddedWebappClassLoader`. `DispatcherServlet` is registered via `DispatcherServletAutoConfiguration` which creates a `DispatcherServletRegistrationBean` -- a Spring wrapper around `ServletRegistrationBean`. The servlet is registered with Tomcat before the connector starts accepting connections.

6. Realistic Practical Example

```
// Switch from Tomcat to Undertow -- change one dependency:
// In pom.xml, exclude tomcat from starter-web and add:
// <dependency>
//   <groupId>org.springframework.boot</groupId>
//   <artifactId>spring-boot-starter-undertow</artifactId>
// </dependency>
// Boot detects UndertowServletWebServerFactory on classpath;
// TomcatServletWebServerFactory's @ConditionalOnClass fails -> Undertow wins

// Custom error page via TomcatServletWebServerFactory:
@Bean
public TomcatServletWebServerFactory servletContainer() {
    TomcatServletWebServerFactory factory = new TomcatServletWebServerFactory();
    factory.addErrorPages(new ErrorPage(HttpStatus.NOT_FOUND, "/404"));
    return factory;
}
```

7. Common Mistakes and Misconceptions

Mistake: Packaging as a WAR and deploying to external Tomcat while also having embedded Tomcat on the classpath -- use `SpringBootServletInitializer` and mark tomcat scope as 'provided'.

Misconception: The embedded server starts before all beans are wired -- actually, `TomcatWebServer.start()` is called at the end of `refresh()`, after all beans are ready.

Good practice: Use `server.tomcat.threads.max` and `server.tomcat.connection-timeout` properties to tune the embedded server without any Java code.

1. Plain English Explanation

Any library author can ship auto-configuration so that users get zero-config integration. You need three things: a `@AutoConfiguration` class with `@Conditional` guards, a properties POJO annotated with `@ConfigurationProperties`, and a registration file so Spring Boot discovers your class at startup. Users can always override by defining their own beans.

2. Real-World Analogy

Packaging auto-configuration in a library is like bundling a default template with a word processor plugin. The template activates automatically when you install the plugin (classpath detection), but users can replace it with their own template -- the plugin checks if a custom template exists before applying the default.

3. Minimal Working Example

```
// Step 1: Properties POJO
@ConfigurationProperties(prefix = "mylib")
public class MyLibProperties {
    private String apiKey = "default-key";
    private int retries = 3;
    // getters + setters
}

// Step 2: Auto-configuration class
@AutoConfiguration
@EnableConfigurationProperties(MyLibProperties.class)
@ConditionalOnClass(MyLibClient.class)
public class MyLibAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    public MyLibClient myLibClient(MyLibProperties props) {
        return new MyLibClient(props.getApiKey(), props.getRetries());
    }
}
```

4. Architecture Diagram

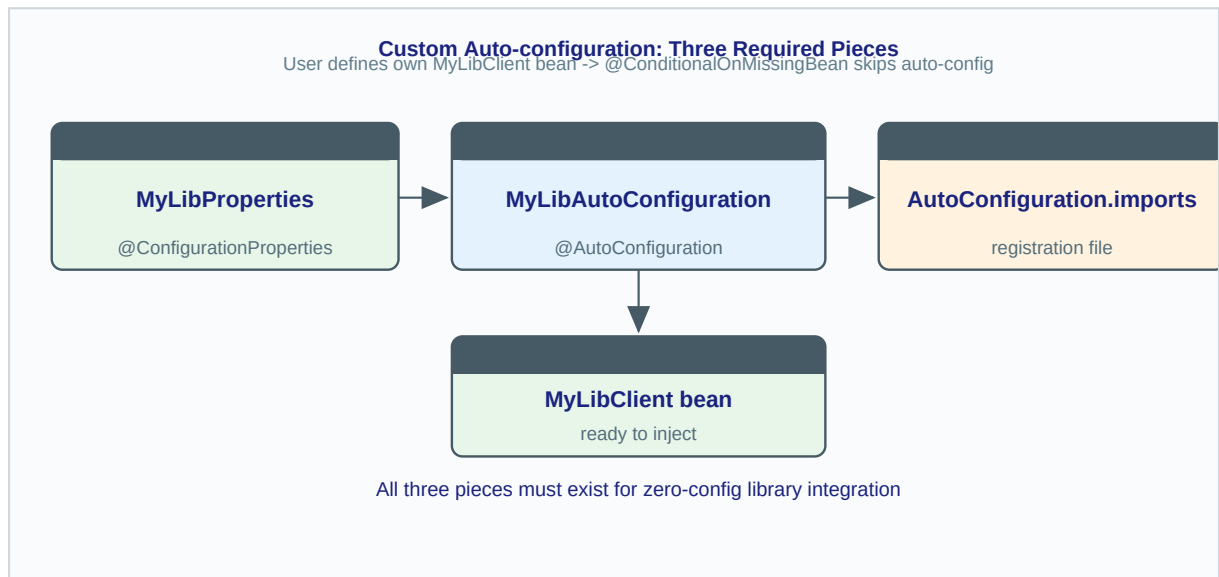


Figure I-1: Three components required for a custom auto-configuration library

5. Behind the Scenes

@AutoConfiguration is a specialization of @Configuration that adds ordering semantics (before/after attributes) and marks the class as auto-configuration so tools can identify it. The .imports registration file must be at exactly META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports inside your library JAR. Spring Boot 3.x uses ImportCandidates.load() to read this file from every JAR on the classpath. @EnableConfigurationProperties registers the properties POJO as a bean and triggers ConfigurationPropertiesBindingPostProcessor to bind Environment properties to it.

6. Realistic Practical Example

```
// Ordering auto-configurations: run after DataSource is ready
@AutoConfiguration(after = DataSourceAutoConfiguration.class)
@ConditionalOnSingleCandidate(DataSource.class)
@ConditionalOnClass(MyAuditService.class)
public class AuditAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    public MyAuditService auditService(DataSource dataSource) {
        return new MyAuditService(dataSource);
    }
    @Bean
    @ConditionalOnMissingBean
    @ConditionalOnProperty("audit.async", havingValue="true")
    public AsyncAuditDecorator asyncAudit(MyAuditService svc) {
        return new AsyncAuditDecorator(svc);
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: Using @Component on an auto-configuration class -- Boot processes auto-config separately from component scan; @Component causes double registration.

Mistake: Not using @ConditionalOnMissingBean -- without it, your auto-config bean conflicts with a user-defined bean of the same type.

Good practice: Always add `after=` ordering on `@AutoConfiguration` to declare dependencies on other auto-configs (e.g., after `DataSourceAutoConfiguration`).

1. Plain English Explanation

Spring Boot Actuator adds production-ready HTTP (and JMX) endpoints that expose operational information about your running application. /health reports whether the app and its dependencies (DB, disk, cache) are up. /metrics exposes hundreds of counters and gauges via Micrometer. /beans lists every bean in the ApplicationContext. These endpoints are auto-configured when you add spring-boot-starter-actuator.

2. Real-World Analogy

Actuator endpoints are the cockpit instrument panel of your application. /health is the engine warning light, /metrics is the fuel gauge and RPM meter, and /beans is the full maintenance log showing every component installed. You would not fly without these instruments, and you should not run production services without Actuator.

3. Minimal Working Example

```
// pom.xml -- add the starter:
// <dependency>
//   <groupId>org.springframework.boot</groupId>
//   <artifactId>spring-boot-starter-actuator</artifactId>
// </dependency>

// application.properties -- expose endpoints:
management.endpoints.web.exposure.include=health,metrics,beans,info
management.endpoint.health.show-details=always

// Custom health indicator:
@Component
public class PaymentApiHealth implements HealthIndicator {
    public Health health() {
        boolean up = pingPaymentApi();
        return up ? Health.up().build()
                : Health.down().withDetail("reason", "timeout").build();
    }
}
```

4. Architecture Diagram

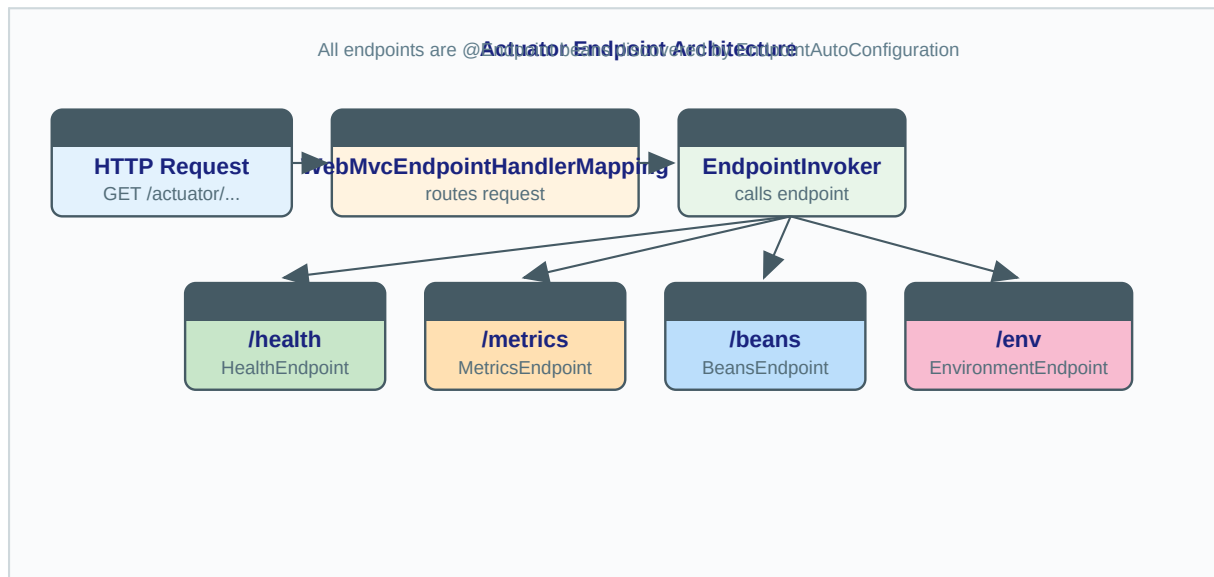


Figure J-1: Actuator request routing from HTTP to individual endpoint implementations

5. Behind the Scenes

ActuatorAutoConfiguration (loaded via `.imports`) registers an `EndpointAutoConfigurationImportSelector` which scans for `@Endpoint`, `@WebEndpoint`, and `@RestControllerEndpoint` beans. `WebMvcEndpointManagementContextConfiguration` creates a `WebMvcEndpointHandlerMapping` that maps `/actuator/{endpoint-id}` URLs to endpoint invoker methods. `HealthEndpoint` aggregates all `HealthIndicator` beans (`DataSourceHealthIndicator`, `DiskSpaceHealthIndicator`, plus your custom ones) via `CompositeHealthContributor`. `MetricsEndpoint` delegates to a `MeterRegistry` (backed by `Micrometer`) which records JVM metrics, HTTP server metrics, and custom counters. `BeansEndpoint` simply calls `ApplicationContext.getBeanDefinitionNames()` and returns the result as JSON.

6. Realistic Practical Example

```
// Custom Micrometer metric:
@RestController
public class OrderController {
    private final Counter orderCounter;

    public OrderController(MeterRegistry registry) {
        this.orderCounter = Counter.builder("orders.created")
            .description("Total orders created")
            .tag("region", "eu-west")
            .register(registry);
    }

    @PostMapping("/orders")
    public ResponseEntity<Order> create(@RequestBody Order order) {
        orderCounter.increment();
        return ResponseEntity.ok(order);
    }
}

// Visible at: GET /actuator/metrics/orders.created
```

7. Common Mistakes and Misconceptions

Mistake: Exposing all Actuator endpoints on the public internet -- always secure with Spring Security or restrict to a management port (`management.server.port=8081`).

Misconception: `/health` returning UP means everything is fine -- the default health check only verifies the app started; you must add custom `HealthIndicators` for your dependencies.

Good practice: Use `management.endpoint.health.group.*` to define named health groups (liveness, readiness) for Kubernetes probes -- Boot provides liveness and readiness groups out of the box.

Spring Boot Auto-configuration -- Quick Reference

Annotation / Class	Purpose	Key Internal Class
@SpringBootApplication	Meta-annotation: Config+Scan+AutoConfig	AnnotationConfigUtils
@EnableAutoConfiguration	Triggers auto-config import selector	AutoConfigurationImportSelector
AutoConfiguration.imports	Registry of all auto-config classes	ImportCandidates.load()
@ConditionalOnClass	Activate if class on classpath	OnClassCondition + ClassUtils.isPresent()
@ConditionalOnMissingBean	Skip if user bean exists	OnBeanCondition
@ConditionalOnProperty	Activate if property set	OnPropertyCondition
DataSourceAutoConfiguration	Creates DataSource (Hikari default)	DataSourceConfiguration.Hikari
@ConfigurationProperties	Binds property prefix to POJO	ConfigurationPropertiesBindingPostProcessor
SpringApplication.run()	Six-phase startup orchestrator	AbstractApplicationContext.refresh()
TomcatServletWebServerFactory	Creates embedded Tomcat	ServletWebServerApplicationContext.onRefresh()
@AutoConfiguration	Custom auto-config marker	.imports registration file
HealthEndpoint	Aggregates HealthIndicators	CompositeHealthContributor
MetricsEndpoint	Exposes Micrometer meters	MeterRegistry

Key Rules

- Always keep @SpringBootApplication at the root package so @ComponentScan finds all beans.
- Use @ConditionalOnClass + @ConditionalOnMissingBean together -- the Boot safety pattern.
- Register custom auto-config via META-INF/spring/AutoConfiguration.imports (not spring.factories in Boot 3+).
- Prefer @ConfigurationProperties over @Value for groups of related properties.
- Secure Actuator endpoints -- never expose /beans or /env on a public port.
- Use ApplicationRunner (not @PostConstruct) for post-startup logic needing fully wired beans.